

HDOC Day-19

Implementation of SGPA and CGPA Calculation Using Nested Loops & Decision-Making Constructs in C

Problem Statement

The algorithm for calculating the Semester Grade Point Average (SGPA) and Cumulative Grade Point Average (CGPA) for m semesters is given below. Each semester has n courses, where n may vary between semesters. Assume that the student has passed all courses. Prepare a test-case table covering valid, invalid, and boundary inputs; write the C program based on the given algorithm; compile and execute it; and test it against the prepared test cases.

Algorithm

1. Start.
2. Define the variables and their types: `global_id`, `m`, `n`, `semester`, `course`, `code`, `credit`, `GP`, `semester_TC`, `semester_TGC`, `cumulative_TC`, and `cumulative_TGC` as integers; `grade` as a character; and `SGPA` and `CGPA` as floating-point variables.
3. Get `global_id`.
4. Validate `global_id`: If `global_id` is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
5. Get `m`.
6. Validate `m`: If `m` is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
7. Set the cumulative values: `cumulative_TC = 0`, `cumulative_TGC = 0`
8. Repeat Steps 8.1 to 8.8 for m semesters:
 - 8.1. Set the semester totals:
$$\text{semester_TC} = 0, \text{semester_TGC} = 0$$
 - 8.2. Get `n`.
 - 8.3. Validate `n`: If `n` is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
 - 8.4. Repeat Steps 8.4.1 to 8.4.9 for n courses:
 - 8.4.1. Get `code`.
 - 8.4.2. Validate `code`: If it is not positive, display an error message and request it again. Repeat until a positive value is entered.
 - 8.4.3. Get `credit`.
 - 8.4.4. Validate `credit`: If it is not positive, display an error message and request it again. Repeat until a positive value is entered.

8.4.5. Get grade.

8.4.6. Validate grade: Accept only O, A, B, C, D, E, or P. If the grade is invalid, display an error message and request it again. Repeat until a valid grade is entered.

8.4.7. Find GP corresponding to grade using the university's conversion scale:

$$O = 10, A = 9, B = 8, C = 7, D = 6, E = 5, P = 4$$

8.4.8. Calculate the semester total grade credits:

$$\text{semester_TGC} = \text{semester_TGC} + (\text{GP} * \text{credit})$$

8.4.9. Calculate the semester total credits:

$$\text{semester_TC} = \text{semester_TC} + \text{credit}$$

8.5. Calculate the SGPA of the current semester:

$$\text{SGPA} = \text{semester_TGC} / \text{semester_TC}$$

8.6. Display semester and SGPA.

8.7. Add the semester grade credits to the cumulative grade credits:

$$\text{cumulative_TGC} = \text{cumulative_TGC} + \text{semester_TGC}$$

8.8. Add the semester credits to the cumulative credits:

$$\text{cumulative_TC} = \text{cumulative_TC} + \text{semester_TC}$$

9. Calculate the CGPA:

$$\text{CGPA} = \text{cumulative_TGC} / \text{cumulative_TC}$$

10. Display global_id, m, cumulative_TC, and CGPA.

11. Stop.

Test Case Table

Case	Type	Input / Condition	Expected Result
1	Valid	Global ID=12345, m=2; Sem 1: n=3: (101,4,A), (102,3,O), (103,3,B); Sem 2: n=3: (201,4,B), (202,4,A), (203,2,O)	Sem 1 SGPA = 9.00; Sem 2 SGPA = 8.80; CGPA = 8.90
2	Invalid	Global ID=-10, then 12345; m=0, then 1; n=-2, then 1; code=0, then 101; credit=-3, then 4; grade=F, then A	Error displayed for each invalid input and input requested again; SGPA = 9.00; CGPA = 9.00
3	Boundary	Global ID=1, m=1, n=1, code=1, credit=1, grade=P	SGPA = 4.00 and CGPA = 4.00

C Program

```
#include <stdio.h>

int main()
{
    int global_id, m, n, semester, code, credit, GP, i;
    int semester_TC, semester_TGC;
    int cumulative_TC = 0, cumulative_TGC = 0;
    float SGPA, CGPA;
    char grade;

    printf("Enter Global ID: ");
    scanf("%d", &global_id);

    while(global_id <= 0)
    {
        printf("Invalid! Enter again: ");
        scanf("%d", &global_id);
    }

    printf("Enter number of semesters: ");
    scanf("%d", &m);

    while(m <= 0)
    {
        printf("Invalid! Enter again: ");
        scanf("%d", &m);
    }

    for(semester = 1; semester <= m; semester++)
    {
        semester_TC = 0;
        semester_TGC = 0;

        printf("\nSemester %d\n", semester);
        printf("Enter number of courses: ");
        scanf("%d", &n);

        while(n <= 0)
        {
            printf("Invalid! Enter again: ");
            scanf("%d", &n);
        }

        for(i = 1; i <= n; i++)
        {
            printf("\nCourse %d\n", i);

            printf("Enter course code: ");
            scanf("%d", &code);

            while(code <= 0)
            {
                printf("Invalid! Enter again: ");
                scanf("%d", &code);
            }

            printf("Enter credits: ");
            scanf("%d", &credit);

            while(credit <= 0)
            {
                printf("Invalid! Enter again: ");
                scanf("%d", &credit);
            }
        }
    }
}
```

```

printf("Enter grade: ");
scanf(" %c", &grade);

while(grade!='O' && grade!='A' && grade!='B' &&
      grade!='C' && grade!='D' && grade!='E' &&
      grade!='P')
{
    printf("Invalid! Enter again: ");
    scanf(" %c", &grade);
}

switch(grade)
{
    case 'O': GP = 10; break;
    case 'A': GP = 9;  break;
    case 'B': GP = 8;  break;
    case 'C': GP = 7;  break;
    case 'D': GP = 6;  break;
    case 'E': GP = 5;  break;
    case 'P': GP = 4;  break;
}

semester_TGC = semester_TGC + GP * credit;
semester_TC = semester_TC + credit;
}

SGPA = (float)semester_TGC / semester_TC;
printf("SGPA = %.2f\n", SGPA);

cumulative_TGC = cumulative_TGC + semester_TGC;
cumulative_TC = cumulative_TC + semester_TC;
}

CGPA = (float)cumulative_TGC / cumulative_TC;

printf("\nGlobal ID = %d\n", global_id);
printf("Number of Semesters = %d\n", m);
printf("Cumulative TC = %d\n", cumulative_TC);
printf("CGPA = %.2f\n", CGPA);

return 0;
}

```

Compilation

```
gcc day19.c -o day19
```



```

(kali@kali)-[~/hdoc19]
$ gcc day19.c -o day19

```

Execution

./day19

```
(kali㉿kali)-[~/hdoc19]
$ ./day19
Enter Global ID: 12345
Enter number of semesters: 2

Semester 1
Enter number of courses: 3

Course 1
Enter course code: 101
Enter credits: 4
Enter grade: A

Course 2
Enter course code: 102
Enter credits: 3
Enter grade: O

Course 3
Enter course code: 103
Enter credits: 3
Enter grade: B
SGPA = 9.00

Semester 2
Enter number of courses: 3

Course 1
Enter course code: 201
Enter credits: 4
Enter grade: B

Course 2
Enter course code: 202
Enter credits: 4
Enter grade: A

Course 3
Enter course code: 203
Enter credits: 2
Enter grade: O
SGPA = 8.80

Global ID = 12345
Number of Semesters = 2
Cumulative TC = 20
CGPA = 8.90
```

```
(kali㉿kali)-[~/hdoc19]
$ ./day19
Enter Global ID: -10
Invalid! Enter again: 12345
Enter number of semesters: 0
Invalid! Enter again: 1

Semester 1
Enter number of courses: -2
Invalid! Enter again: 1

Course 1
Enter course code: 0
Invalid! Enter again: 101
Enter credits: -3
Invalid! Enter again: 4
Enter grade: F
Invalid! Enter again: A
SGPA = 9.00

Global ID = 12345
Number of Semesters = 1
Cumulative TC = 4
CGPA = 9.00
```

```
(kali㉿kali)-[~/hdoc19]
$ ./day19
Enter Global ID: 1
Enter number of semesters: 1

Semester 1
Enter number of courses: 1

Course 1
Enter course code: 1
Enter credits: 1
Enter grade: P
SGPA = 4.00

Global ID = 1
Number of Semesters = 1
Cumulative TC = 1
CGPA = 4.00
```

Result: The SGPA and CGPA calculation program was implemented using nested loops and decision-making constructs in C. The program validates the required inputs, calculates SGPA for each semester, maintains cumulative credits and grade credits, and calculates the final CGPA.